PROJECT INITIATION DOCUMENT (PID) • BIG DATA & DISTRIBUTED SYSTEMS

AutoData Lake: Distributed Object Storage, Consistent Hashing & Telemetry Lakehouse

A 4-Week Rapid Engineering Term Project for Connected Vehicle Fleets • Fall 2026

EXECUTIVE FORMULATION: Connected vehicle fleets generate massive streams of high-velocity sensor data alongside heavy unstructured binary assets (service manuals, diagnostic trouble codes, and cluster scans). This 4-week term project implements foundational Big Data and distributed systems architectures from first principles—bypassing monolithic relational databases and superficial API wrappers. The architecture integrates **Distributed Object Storage (MinIO S3)**, **Content-Addressable Storage (CAS via SHA-256)** for multi-gigabyte deduplication, a custom **Consistent Hashing Ring with Virtual Nodes** for fault-tolerant partition sharding, an in-memory **Bloom Filter** for sub-millisecond DTC pre-indexing, and an **Apache Parquet Lakehouse** analyzed via **DuckDB/PySpark**. The delivery is scheduled across four intensive, milestone-driven weekly sprints.

Course & Degree:	Big Data Systems (CS/DS) — Bachelor of Science in Data Science (BSDS)	**Execution Window:**	**4 Weeks (Intensive Term Sprint)**
Department:	Department of Data Science / Computing	**Submission Category:**	Term Project Initiation Document (PID)

1. Project Team Roster & Core Technical Roles

Sr.	Member Name	Roll No.	Assigned Technical Role (4-Week Scope)
1	**Talha Ali Bhatti**	**bsdsf23a016**	Distributed Storage & Streaming Lead: MinIO S3 cluster setup, bucket lifecycle policies, OBD-II stream generator, and Apache Parquet partitioning.
2	**Muhammad Abdullah**	**bsdsf23a032**	Data Engineering & Analytics Lead: Batch ETL pipelines (DuckDB/PySpark), fleet anomaly models, and query latency benchmarking.
3	**Abdullah**	**bsdsf23a035**	Distributed Algorithms Lead: Consistent Hashing Ring (Virtual Nodes), Content-Addressable Storage (CAS SHA-256), and Bloom Filter engine.

2. Big Data Characteristics (The 5 V's of Automotive Telematics)

Dimension	Workload Manifestation in Connected Vehicles	Architectural Solution in AutoData Lake
Volume	10,000+ vehicles generating ~50MB daily telemetry + 20GB unstructured OEM service manual PDFs & damage photos.	Distributed Object Storage (MinIO S3) decoupled from compute; storage compression via Apache Parquet.
Velocity	Continuous CAN-bus & OBD-II sensor streams arriving at 10 Hz to 50 Hz per vehicle requiring sub-second triage.	Asynchronous Dual-Speed Ingestion (Speed Buffer) utilizing sliding-window micro-batching.
Variety	Structured telemetry (Parquet), Unstructured binary media (PDF manuals, cluster photos), and Semi-structured logs (DTCs).	Unified Multi-Modal Lakehouse storing structured time-series and binary blobs with unified catalogs.
Veracity	Sensor dropouts, noise, corrupt frames, and redundant manual uploads across owners.	SHA-256 cryptographic verification for Content-Addressable Storage (CAS) and schema enforcement.
Value	Driver safety triage, fleet predictive failure warning, and localized Pakistani PKR repair cost benchmarks.	Distributed OLAP analytics (DuckDB/PySpark) driving component longevity models & price protection.

3. Core Distributed Systems Concepts Implemented

3.1 Distributed Object Storage (MinIO S3 API): Eliminates POSIX inode bottlenecks by persisting unstructured assets as immutable objects under partitioned URI keys: *s3://autodata-lake/telemetry/year=2026/make=toyota/model=corolla/vin={VIN}/*.

3.2 Content-Addressable Storage (CAS SHA-256): Ingested manuals and cluster photos are indexed by their cryptographic hash *H(F) = SHA-256(F)*. Duplicates share a single physical block with atomic reference counters, saving 40%–60% storage.

3.3 Consistent Hashing Ring (Ketama with Virtual Nodes): Vehicle VINs are mapped onto a 32-bit keyspace [0, 2^32 - 1] with *V = 150* virtual replicas per node. Adding or removing a node migrates only *K/N* keys on average, avoiding full-cluster invalidation.

3.4 Scalable In-Memory Bloom Filter: Pre-indexes 10,000+ SAE diagnostic trouble codes (DTCs) using $k = (m/n) \ln(2)$ hash functions, guaranteeing zero false negatives and intercepting non-existent queries prior to disk I/O.

3.5 Columnar Lakehouse Storage (Apache Parquet + Snappy): High-velocity telemetry is converted into columnar format with dictionary encoding and run-length compression, achieving >70% size reduction and fast column projection pushdown.

4. 4-Week Rapid Execution Roadmap & Sprint Milestones

The project is structured into four intensive 1-week sprints to achieve full end-to-end implementation:

Sprint	Timeline	Sprint Scope & Key Technical Deliverables	Owner / Lead
W1	**Week 1** **Days 1–7**	**Infrastructure & CAS Deduplication Engine:** • Containerized MinIO Distributed Object Storage setup with S3 bucket hierarchy. • Implement Content-Addressable Storage (CAS SHA-256) deduplication module. • Ingest OEM PDF service manuals and benchmark storage deduplication ratio.	**Talha Ali (016)** **Abdullah (035)**
W2	**Week 2** **Days 8–14**	**Consistent Hashing & Bloom Filter Layer:** • Implement *ConsistentHashRing* with virtual node replication (V=150) and bisect routing. • Implement in-memory *BloomFilter* for 10,000+ SAE OBD-II diagnostic fault codes. • Run statistical Chi-Square test on hash ring to verify uniform key distribution.	**Abdullah (035)**
W3	**Week 3** **Days 15–21**	**Stream Ingestion & Columnar Lakehouse Analytics:** • Build simulated high-frequency CAN-bus / OBD-II stream generator (10 Hz – 50 Hz). • Implement streaming buffer and compaction into partitioned Apache Parquet files. • Configure DuckDB / PySpark analytical queries for fleet-wide anomaly detection.	**Talha Ali (016)** **M. Abdullah (032)**
W4	**Week 4** **Days 22–28**	**Integration, Empirical Benchmarking & Project Defense:** • Connect data engine with FastAPI endpoints and cluster telemetry dashboard. • Execute 5 quantitative benchmarks (Chi-Square uniformity, K/N churn, query latency). • Finalize technical term project report, code profiling, and presentation slide deck.	**All Members** **(Joint Defense)**

5. 4-Week RACI Responsibility Assignment Matrix

Technical Work Package	Talha Ali (016)	M. Abdullah (032)	Abdullah (035)
Week 1: MinIO Object Store Setup & S3 Buckets	**Accountable (A)**	Informed (I)	Consulted (C)
Week 1: CAS Deduplication Engine (SHA-256)	Informed (I)	Consulted (C)	**Accountable (A)**
Week 2: Consistent Hashing Ring with Virtual Nodes	Consulted (C)	Informed (I)	**Accountable (A)**
Week 2: Scalable In-Memory Bloom Filter Lookup	Informed (I)	Consulted (C)	**Accountable (A)**
Week 3: OBD-II High-Frequency Stream Ingestion	**Accountable (A)**	Consulted (C)	Informed (I)
Week 3: Apache Parquet Columnar Lakehouse Storage	**Accountable (A)**	Consulted (C)	Consulted (C)
Week 3: Distributed Fleet Analytics (DuckDB/PySpark)	Consulted (C)	**Accountable (A)**	Informed (I)
Week 4: Empirical Benchmarks & Chi-Square Testing	Consulted (C)	**Accountable (A)**	Consulted (C)
Week 4: Final Documentation, Term Report & Defense	Responsible (R)	Responsible (R)	Responsible (R)

6. Academic Significance & Evaluation Defensibility

By delivering AutoData Lake within this disciplined 4-week timeline, the group presents a concrete systems-level engineering project that directly answers academic evaluation criteria: (1) Core distributed algorithms are implemented from first principles rather than relying on black-box AI code; (2) Performance is validated through formal statistical tests (Chi-Square uniformity, K/N re-sharding overhead, and query latency); and (3) The architecture mirrors state-of-the-art telematics pipelines deployed in the automotive industry.

Submitted on behalf of Group BSDS-23-AutoData: Talha Ali Bhatti (bsdsf23a016) • Muhammad Abdullah (bsdsf23a032) • Abdullah (bsdsf23a035)